Simulando transferência de calor em GPU com OpenMP

Eugenio Vitor Lopes dos Santos

DCA3703, Programação Paralela, Universidade Federal do Rio Grande do Norte

1 Enunciado

Faça o exercício de transferência de calor (`heat.c`, slide 64) do tutorial de programação de GPUs com OpenMP. Explore as diretivas de paralelização e movimentação de dados para copiar dados entre host e dispositivo. Experimente com diferentes tamanhos de problema perfilando o programa com o `nsys`.

2 Fundamentação Teórica

Nesta tarefa, OpenMP encaminha o cálculo à GPU e controla as transferências de dados entre host e dispositivo.

A diretiva `#pragma omp target` define uma região executada em um dispositivo. Suas principais cláusulas são:

Cláusula de `target`	Papel
`map`	Mapeia dados entre hospedeiro e dispositivo. `to` envia valores; `from` devolve valores; `tofrom` faz ambos; `alloc` reserva espaço sem copiar os valores.
`device`	Seleciona o dispositivo em que a região será executada.
`private`	Cria uma instância privada da variável, sem copiar seu valor de entrada.
`firstprivate`	Cria uma instância privada inicializada com o valor de entrada.
`nowait`	Permite adiar a execução da tarefa de dispositivo, sem a espera imediata do hospedeiro; dependências precisam ser respeitadas.

Tabela 1: Cláusulas de `target`; o stencil utiliza explicitamente `map`.

A diretiva `#pragma omp loop` permite a execução concorrente das iterações. `collapse(2)` associa dois laços à distribuição de trabalho; `reduction` combina resultados parciais, como em uma soma. O stencil requer apenas `collapse(2)`, pois cada iteração escreve uma célula distinta.

3 Experimento

3.1 Modelo e código

O `heat.c` simula a difusão de calor em um quadrado de lado 1000, com coeficiente de difusão 0,1, bordas nulas e temperatura inicial senoidal, até o instante 0,5.

A função `solve` usa um stencil de cinco pontos: lê a célula e seus quatro vizinhos em `u` e escreve a próxima temperatura em `u_tmp`. Os campos distintos evitam misturar valores de passos diferentes.

A atualização é executada na GPU, e a norma L2 compara o campo final com a solução analítica do código base.

3.2 Paralelização e cópias dos campos

A atualização da malha usa:

```
#pragma omp target teams loop collapse(2) \
    map(to:u[0:cells]) map(from:u_tmp[0:cells])
```

`target` encaminha a região ao dispositivo, e `teams loop collapse(2)` distribui as n^2 atualizações entre equipes e threads da GPU. Por exemplo, uma malha 4×4 oferece 16 células independentes para distribuir.

`map(to:u[0:cells])` envia o campo atual; `map(from:u_tmp[0:cells])` devolve o resultado, sem enviar os valores anteriores do campo de saída, que será sobrescrito. A seção `[0:cells]` mapeia os n^2 elementos apontados por cada ponteiro. Há duas cópias por passo.

Sem `nowait`, a região termina antes da troca dos ponteiros na CPU. O campo calculado torna-se a entrada do próximo passo, respeitando a dependência temporal, sem uma cópia adicional dos elementos.

3.3 Configuração e medição

Foram medidas malhas de 256^2, 1024^2 e 4096^2 células, com 100 passos, no NPAD (partição `gpu-8-v100`), usando uma Tesla V100-SXM2 de 16 GB e um Intel Xeon E5-2683 v4. A compilação utilizou NVIDIA HPC SDK 24.11:

```
nvc -O3 -mp=gpu -Minfo=mp heat.c -lm -o heat_map
OMP_TARGET_OFFLOAD=MANDATORY ./heat_map 1024 100
```

A CPU usou oito threads, com `OMP_PLACES=cores` e `OMP_PROC_BIND=close`; as threads da GPU foram definidas pelo compilador.

Os resultados pertencem à configuração `map` do job 2138636. Para cada malha, apresenta-se a mediana de três execuções sem perfilamento. A medição com `clock_gettime` e `CLOCK_MONOTONIC` inclui os 100 passos, as transferências e a sincronização; exclui inicialização, diagnóstico do dispositivo e norma L2.

Cada malha foi perfilada com o `nsys` em uma execução separada das medições de tempo, registrando os kernels e as transferências entre host e GPU.

4 Resultados

4.1 Desempenho

n	Mediana do tempo (s)
256	0,031428
1024	0,302749
4096	4,489523

Tabela 2: Medianas de três execuções sem perfilamento por malha, com 100 passos e cópias dos campos a cada passo.

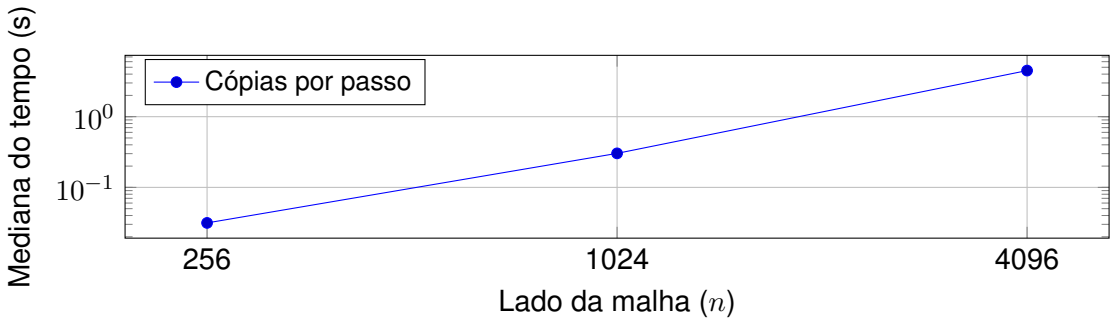


Figura 1: Tempo dos 100 passos, incluindo o envio e o retorno dos campos a cada passo. Os eixos usam escala logarítmica.

O crescimento da malha aumenta o trabalho dos kernels e o volume copiado. O tempo de parede inclui também lançamentos, gerenciamento dos mapeamentos e sincronização.

4.2 Perfilamento de kernels e transferências

n	Kernels (ms)	Cópias (ms)	Cópias	Volume (MiB)
256	0,293	9,816	200	100
1024	2,698	246,590	200	1600
4096	34,582	4483,764	200	25600

Tabela 3: Somas das durações registradas pelo `nsys` em uma execução por malha. Foram considerados os 100 kernels do stencil e as cópias dos campos, excluindo o diagnóstico inicial. O volume soma os dois sentidos.

Os perfis confirmam um kernel, um envio e um retorno por passo. As cópias acumulam mais tempo que os kernels nas três malhas. O volume total cresce com n^2: são $200 \times 8n^2$ bytes para 100 passos em precisão dupla.

As durações perfiladas vêm de execuções distintas das medianas e não incluem todos os custos do runtime e esperas do tempo de parede. A tabela exclui o kernel e as duas pequenas transferências do diagnóstico inicial.

A verificação pela norma L2 apresentou pequenos erros em relação à solução analítica nas três malhas medidas.

5 Conclusão

Na configuração medida, o stencil executou na GPU com pequenos erros em relação à solução analítica. A menor malha teve o menor tempo para uma carga também menor; as cópias por passo foram o principal custo perfilado.

Anexo: Código heat.c

```c
/*
** PROGRAM: heat equation solve
**
** PURPOSE: This program will explore use of an explicit
**          finite difference method to solve the heat
**          equation under a method of manufactured solution (MMS)
**          scheme. The solution has been set to be a simple
**          function based on exponentials and trig functions.
**
**          A finite difference scheme is used on a 1000x1000 cube.
**          A total of 0.5 units of time are simulated.
**
**          The MMS solution has been adapted from
**          G.W. Recktenwald (2011). Finite difference approximations
**          to the Heat Equation. Portland State University.
**
** USAGE:   Run with two arguments:
**          First is the number of cells.
**          Second is the number of timesteps.
**          ./heat 100 10
**
** HISTORY: Written by Tom Deakin, Oct 2018
*/
#include <stdlib.h>
#include <stdio.h>
#include <math.h>
#include <omp.h>

#define PI acos(-1.0)
#define LINE "--------------------\n"

void initial_value(const int n, const double dx, const double length,
                   double *restrict u);
void zero(const int n, double *restrict u);
void solve(const int n, const double alpha, const double dx, const double dt,
           const double *restrict u, double *restrict u_tmp);
double solution(const double t, const double x, const double y,
                const double alpha, const double length);
double l2norm(const int n, const double *restrict u, const int nsteps,
              const double dt, const double alpha, const double dx,
              const double length);

int main(int argc, char *argv[])
{
    double start = omp_get_wtime();
    int n = 1000;
    int nsteps = 10;
    if (argc == 3) {
        n = atoi(argv[1]);
        nsteps = atoi(argv[2]);
    }
    if (n <= 0 || nsteps <= 0) {
        fprintf(stderr, "n e passos devem ser positivos.\n");
        return EXIT_FAILURE;
    }

    double alpha = 0.1;
    double length = 1000.0;
    double dx = length / (n + 1);
```

```
60      double dt = 0.5 / nsteps;
61      double r = alpha * dt / (dx * dx);
62
63      printf("\n MMS heat equation\n\n");
64      printf(LINE);
65      printf("Problem input\n\n");
66      printf(" Grid size: %d x %d\n", n, n);
67      printf(" Cell width: %E\n", dx);
68      printf(" Grid length: %lf x %lf\n\n", length, length);
69      printf(" Alpha: %E\n\n", alpha);
70      printf(" Steps: %d\n", nsteps);
71      printf(" Total time: %E\n", dt * (double)nsteps);
72      printf(" Time step: %E\n", dt);
73      printf(LINE);
74      printf("Stability\n\n");
75      printf(" r value: %lf\n", r);
76      if (r > 0.5)
77          printf(" Warning: unstable\n");
78      printf(LINE);
79
80      size_t cells = (size_t)n * n;
81      double *u = malloc(sizeof(*u) * cells);
82      double *u_tmp = malloc(sizeof(*u_tmp) * cells);
83      double *tmp;
84      if (!u || !u_tmp) {
85          fprintf(stderr, "Falha de alocacao.\n");
86          free(u);
87          free(u_tmp);
88          return EXIT_FAILURE;
89      }
90      initial_value(n, dx, length, u);
91      zero(n, u_tmp);
92
93      double tic = omp_get_wtime();
94      for (int t = 0; t < nsteps; ++t) {
95          solve(n, alpha, dx, dt, u, u_tmp);
96          tmp = u;
97          u = u_tmp;
98          u_tmp = tmp;
99      }
100     double toc = omp_get_wtime();
101     double norm = l2norm(n, u, nsteps, dt, alpha, dx, length);
102     double stop = omp_get_wtime();
103
104     printf("Results\n\n");
105     printf("Error (L2norm): %E\n", norm);
106     printf("Solve time (s): %lf\n", toc - tic);
107     printf("Total time (s): %lf\n", stop - start);
108     printf(LINE);
109     free(u);
110     free(u_tmp);
111 }
112
113 void initial_value(const int n, const double dx, const double length,
114                    double *restrict u)
115 {
116     double y = dx;
117     for (int j = 0; j < n; ++j) {
118         double x = dx;
119         for (int i = 0; i < n; ++i) {
120             u[i + j * n] = sin(PI * x / length) * sin(PI * y / length);
```

```
            x += dx;
        }
        y += dx;
    }
}

void zero(const int n, double *restrict u)
{
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            u[i + j * n] = 0.0;
        }
    }
}

void solve(const int n, const double alpha, const double dx, const double dt,
           const double *restrict u, double *restrict u_tmp)
{
    const double r = alpha * dt / (dx * dx);
    const double r2 = 1.0 - 4.0 * r;
    size_t cells = (size_t)n * n;
    #pragma omp target teams loop collapse(2) \
        map(to:u[0:cells]) map(from:u_tmp[0:cells])
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            u_tmp[i + j * n] = r2 * u[i + j * n]
                + r * ((i < n - 1) ? u[i + 1 + j * n] : 0.0)
                + r * ((i > 0) ? u[i - 1 + j * n] : 0.0)
                + r * ((j < n - 1) ? u[i + (j + 1) * n] : 0.0)
                + r * ((j > 0) ? u[i + (j - 1) * n] : 0.0);
        }
    }
}

double solution(const double t, const double x, const double y,
                const double alpha, const double length)
{
    return exp(-2.0 * alpha * PI * PI * t / (length * length))
        * sin(PI * x / length) * sin(PI * y / length);
}

double l2norm(const int n, const double *restrict u, const int nsteps,
              const double dt, const double alpha, const double dx,
              const double length)
{
    double time = dt * (double)nsteps;
    double l2norm = 0.0;
    double y = dx;
    for (int j = 0; j < n; ++j) {
        double x = dx;
        for (int i = 0; i < n; ++i) {
            double answer = solution(time, x, y, alpha, length);
            l2norm += (u[i + j * n] - answer) * (u[i + j * n] - answer);
            x += dx;
        }
        y += dx;
    }
    return sqrt(l2norm);
}
```